

Deploying Athwart Loop on the Freemium Stack

Vercel (Frontend) · Render (Backend) · Supabase (Database + Storage)

Prepared for: 5-user sample/demo deployment

1. Architecture Overview

Three independent free-tier services, each owning one layer of the stack:

Layer	Service	Role
Frontend	Vercel (Hobby)	Serves the React/Vite build, handles SPA routing
Backend API	Render (Free Web Service)	Runs the Express/Node API, Prisma client, cron jobs, sockets
Database	Supabase (Free Project)	Managed Postgres — replaces local docker-compose db
File Storage	Supabase Storage (Free bucket)	Replaces local disk uploads/ (avatars, documents, attachments)

Request flow: Browser → Vercel (static assets) → Render API (HTTPS, Bearer JWT) → Supabase Postgres (Prisma) + Supabase Storage (file uploads).

2. Deployment Steps (in order)

Deploy in this order — each later step needs a URL/credential from the one before it.

Step 1 — Supabase (Database + Storage)

- Create a new Supabase project (free tier). Note the generated Postgres connection string.
- In Supabase → Settings → Database, copy the **connection pooling** URL (port 6543, pgbouncer) rather than the direct connection — Render's free plan works better through the pooler since it avoids exhausting Postgres' limited direct connection slots.
- Create a Storage bucket (e.g. uploads) with sub-folders/paths for avatar, document, post_attachment — mirroring the current local uploads/ structure.
- Run `npx prisma migrate deploy` once, locally, pointed at the Supabase connection string, to create all tables before the backend ever starts.

Step 2 — Render (Backend API)

- Create a new **Web Service** on Render, connect the GitHub repo, root directory backend/.
- Build command: `npm ci && npm run build` Start command: `npm start`.
- Set environment variables: `DATABASE_URL` (Supabase pooled URL), `JWT_SECRET`, `GROQ_API_KEY`, `PORT` (Render sets this automatically — read `process.env.PORT`, don't hardcode 4000).

- Do **not** run `prisma migrate deploy` inside the build step — this repo already hit a build failure from that (see commits `df0b0b2 / c43c831`). Run migrations manually from your machine against the Supabase URL, or via Render's one-off Shell after deploy.
- Free tier has **no persistent disk** — local file uploads will vanish on every redeploy/restart. This is why Storage must move to Supabase Storage before going live (see Section 4).

Step 3 — Vercel (Frontend)

- Import the repo into Vercel, set root directory to `frontend/` (Framework preset: Vite).
 - Set `VITE_API_URL` to the deployed Render URL, e.g. `https://your-service.onrender.com/api`.
 - `vercel.json` already has the SPA rewrite rule needed for client-side routing — no change required.
 - Note: Vercel's free **Hobby** plan is licensed for personal/non-commercial use per their Terms of Service. Fine for an internal demo with 5 known users; if this becomes a paid/commercial product later, Vercel Pro (\$20/mo) is the correct tier to move to.
-

3. Configuration to Fix Before Deploying

- **CORS is wide open** — `backend/src/app.ts:21` sets `cors({ origin: '*', credentials: true })`. Auth uses Bearer tokens (not cookies), so this won't break login, but it means any website can call your API from a browser. Restrict origin to your Vercel domain before sharing this beyond the 5 test users.
 - **File storage provider** — `src/services/storage/local.storage.provider.ts` writes to local disk. Needs a Supabase-Storage-backed provider swap (or S3-compatible) since Render free has no persistent disk.
 - **Cron reliability** — `src/cron/sla.cron.ts` runs in-process via `node-cron`. It will not fire while the Render free service is asleep (see Section 5). Acceptable for a 5-user demo; revisit if SLA timing becomes important.
-

4. How Much Will Freemium Handle?

For **5 users running this as a sample/demo product**, all three free tiers are comfortably sufficient. Data volume and traffic at this scale are a negligible fraction of any limit below — this is not a capacity risk. The numbers exist mainly so you know when (if ever) to revisit them.

Rough Post-table math (measured against this schema)

Each Post row (title + description + FKs + enums + its 9 indexes) is roughly 500 bytes–1 KB. Supabase's 500MB free database budget is shared across Posts, Comments, Reactions, Mentions, Notifications, and AuditLog rows — but even accounting for that, a free project comfortably holds on the order of **tens of thousands to 100k+ posts** before database size becomes a concern.

The tighter ceiling is **Supabase Storage's 1GB free bucket**. If posts commonly carry a 1–3MB image or PDF attachment, that caps out around **300–1,000 attachments total** — not posts. At 5 users this is a non-issue for a long time; flag it only if attachment usage becomes heavy.

5. Free Tier Limits — Quick Reference

Approximate values as of mid-2026; always confirm current numbers on each provider's pricing page before making a go/no-go call for a real launch.

Service	Limit	Value (Free Tier)	Relevant to this project
Render	Compute	512MB RAM / 0.1 shared vCPU	Fine for 5 users, low concurrency
Render	Sleep/Spin-down	Sleeps after 15 min idle	~30-50s cold start on first request after idle
Render	Persistent disk	None on free plan	Local file uploads are wiped on restart/redeploy
Render	Bandwidth	~100GB/month	Non-issue at this scale
Supabase	Database size	500MB	Tens of thousands+ of posts before this matters
Supabase	File storage	1GB	Real ceiling if attachments/images are used heavily
Supabase	Bandwidth (egress)	~5GB/month	Non-issue at 5 users
Supabase	Project pause	Auto-pauses after 7 days no activity	Needs a keep-alive ping or manual unpause
Vercel	Bandwidth	100GB/month	Non-issue at this scale
Vercel	Usage terms	Hobby = personal/non-commercial use	OK for internal demo; revisit if monetized

6. Pre-Launch Checklist

- ☐ Supabase project created, connection pooling URL copied
- ☐ `prisma migrate deploy` run manually against Supabase (not in Render build step)
- ☐ Supabase Storage bucket created; local storage provider swapped for Supabase-backed one
- ☐ Render Web Service deployed with correct env vars; confirm PORT read from environment
- ☐ Vercel frontend deployed with `VITE_API_URL` pointing at Render
- ☐ CORS origin restricted from `'*'` to the Vercel domain
- ☐ Optional: uptime pinger (UptimeRobot / cron-job.org) hitting a health endpoint every ~10 min, if avoiding cold-start delay matters for the demo

Net takeaway: for a 5-user sample deployment, freemium across Vercel + Render + Supabase is the right call. No paid tier is needed to handle this scale — the fixes above are about correctness and reliability (uploads surviving restarts, cron timing, CORS hygiene), not capacity.